

BookHub — Complete Technical Documentation & Comprehensive Interview Master Guide

Every Tech Stack | Every File | Every Function | Every Line of Code | Every Workflow | Every Database Query — Fully Explained

Project: BookHub Online Bookstore	Backend: Java 21 / Spring Boot 4.0.5
Database: MySQL 8.x + Spring Data JPA / Hibernate ORM	Frontend: Vanilla HTML5, CSS3, JavaScript Fetch API
Architecture: 3-Tier Layered MVC REST	Build Tool: Apache Maven (pom.xml)
JDBC Driver: mysql-connector-j	Connection Pool: HikariCP (auto-configured)
Server: Embedded Apache Tomcat (port 8080)	Serialization: Jackson JSON ObjectMapper

Table of Contents

- Section 1 — Tech Stack: Interview Answers ('I used X to do Y')
- Section 2 — Project File Structure & What Each File Does
- Section 3 — How the Application Accesses MySQL (Complete Database Pipeline)
- Section 4 — How Frontend Connects to Backend (Complete HTTP Lifecycle)
- Section 5 — File-by-File & Function-by-Function Code Breakdown (With Full Code)
- Section 6 — End-to-End Workflow: What Happens When Each Action Is Selected
- Section 7 — Master REST API Reference Table & SQL Query Map
- Section 8 — CRUD Operations Explained With Code
- Section 9 — Exception Handling Pipeline
- Section 10 — Interview Closing Summary & Golden Pitch

Section 1 — Technology Stack: Interview Answers

For each technology used in BookHub, memorize this first-person answer format:

1. Java 21 (Programming Language)

"I used Java 21 as the primary programming language for my BookHub backend. Java is an object-oriented, strongly typed language that runs on the Java Virtual Machine (JVM). I chose Java 21 because it supports modern features like records, pattern matching, and virtual threads, and it provides high performance and cross-platform compatibility."

2. Spring Boot 4.0.5 (Backend Framework)

"I used Spring Boot 4.0.5 as the backend framework to build production-ready REST APIs rapidly. Spring Boot provides auto-configuration — it automatically detects dependencies like Spring Data JPA and MySQL Connector on the classpath and configures DataSource, EntityManager, and Hibernate without me writing XML or boilerplate. It also bundles an embedded Apache Tomcat server so I don't need to install a separate web server."

3. Spring Data JPA (Data Access Abstraction)

"I used Spring Data JPA to implement the data persistence layer. Instead of writing manual JDBC code with Connection, PreparedStatement, and ResultSet, I simply created Java interfaces (BookRepository, CartRepository, OrderRepository) that extend JpaRepository. Spring Data JPA automatically generates the SQL implementation at runtime using dynamic proxies."

4. Hibernate ORM (Object-Relational Mapping Engine)

"I used Hibernate as the ORM provider underneath Spring Data JPA. Hibernate automatically maps my Java entity classes (Book.java, CartItem.java, Order.java) to MySQL database tables. When I annotate a class with @Entity, Hibernate creates or updates the corresponding table. When I call repository.save(), Hibernate generates the INSERT or UPDATE SQL query automatically."

5. MySQL 8.x (Relational Database)

"I used MySQL as the relational database to persistently store all application data. I have three tables: 'book' (catalog inventory), 'cart_item' (temporary shopping cart state with a foreign key to book), and 'orders' (permanent purchase transaction history). MySQL provides ACID transactions, foreign key integrity, and efficient indexed queries."

6. MySQL Connector/J — mysql-connector-j (JDBC Driver)

"I used mysql-connector-j as the JDBC driver dependency in my pom.xml. This is the official MySQL JDBC driver that allows the Java Virtual Machine to establish a TCP network socket connection to the MySQL server running on localhost:3306 and execute SQL commands over that connection."

7. HikariCP (JDBC Connection Pool)

"I used HikariCP as the high-performance JDBC connection pool. It is auto-configured by spring-boot-starter-data-jpa. Instead of opening a new TCP database connection for every single HTTP request (which is very slow), HikariCP maintains a pool of pre-established connections. When a repository method needs to run SQL, it borrows a connection from the pool, executes the query, and returns it to the pool."

8. Spring Web MVC — spring-boot-starter-webmvc (REST Framework)

"I used Spring Web MVC to expose RESTful HTTP endpoints. Spring MVC uses the DispatcherServlet pattern — a front controller that intercepts all incoming HTTP requests, consults handler mappings to find the matching @RestController method, deserializes JSON request bodies using Jackson, invokes the controller method, and serializes the return value back into a JSON HTTP response."

9. Embedded Apache Tomcat (Servlet Container / Web Server)

"I used the embedded Apache Tomcat server that comes bundled with Spring Boot. It starts automatically when I run the application and listens on port 8080 (configured in application.properties). Tomcat accepts TCP connections from web browsers, forwards HTTP requests to Spring's DispatcherServlet, and sends HTTP responses back to the client."

10. REST API (Communication Protocol Pattern)

"I used REST APIs (Representational State Transfer) to create a stateless client-server communication interface. I designed resource-based URL endpoints like /books, /cart, and /orders and mapped them to standard HTTP methods: GET for reading data, POST for creating data, and DELETE for removing data. All data is exchanged in JSON format."

11. JSON (Data Exchange Format) & Jackson (Serializer)

"I used JSON (JavaScript Object Notation) as the standard data format for all communication between the frontend HTML pages and the Spring Boot backend. Jackson ObjectMapper, which is included by default in Spring Boot, automatically converts Java objects to JSON strings (serialization) when sending responses, and converts incoming JSON request bodies back into Java objects (deserialization)."

12. DTO Pattern — ApiResponse<T> (Data Transfer Object)

"I created a generic ApiResponse<T> class as a Data Transfer Object to standardize all API responses. Every endpoint in my application returns the same structure: { "success": true/false, "message": "...", "data": ... }. This ensures the frontend always knows exactly what format to expect, whether the operation succeeded or failed."

13. Vanilla HTML5 & CSS3 (Frontend UI)

"I used plain HTML5 and CSS3 (without React, Angular, or Vue) to build two web pages: index.html (customer storefront) and admin.html (admin dashboard). I used CSS Grid for responsive book card layouts, CSS Flexbox for header alignment, CSS transitions for smooth cart drawer sliding animation, and gradient backgrounds for modern visual design."

14. JavaScript Fetch API (Frontend HTTP Client)

"I used the native JavaScript Fetch API with async/await syntax to send HTTP requests from the browser to my Spring Boot backend REST APIs. Fetch is built into all modern browsers — no external library like Axios is needed. I use fetch() for GET, POST, and DELETE operations and parse the JSON responses to dynamically update the page DOM."

15. CORS — @CrossOrigin (Cross-Origin Resource Sharing)

"I used the @CrossOrigin annotation on all my Spring REST controller classes. CORS is a browser security mechanism that blocks web pages from making requests to a different origin (protocol + host + port). By adding @CrossOrigin, Spring adds the Access-Control-Allow-Origin header to HTTP responses, allowing the browser to accept responses from the backend on port 8080."

16. @ControllerAdvice & @ExceptionHandler (Global Exception Handling)

"I used Spring's @ControllerAdvice annotation to create a GlobalExceptionHandler class that intercepts exceptions thrown by any controller in the application. Instead of returning ugly stack traces or Whitelabel error pages, my handler catches RuntimeExceptions and returns clean JSON error responses with HTTP 400 (Bad Request) or HTTP 500 (Internal Server Error) status codes."

17. Dependency Injection & IoC (Inversion of Control)

"I used Spring's Constructor-Based Dependency Injection throughout the application. Instead of creating objects with the 'new' keyword, Spring's IoC container automatically creates and injects dependencies. For example, BookService receives BookRepository via its constructor, and BookController receives BookService via its constructor. This promotes loose coupling and easy unit testing."

18. MultipartFile & WebMvcConfigurer (File Upload & Static Resource Serving)

"I used Spring's MultipartFile interface to handle binary image uploads from the admin page. When an admin uploads a cover image, the file is received as a MultipartFile, saved to the local uploads/ directory with a unique timestamp prefix, and the public URL is returned. I also configured a WebMvcConfigurer bean to map /uploads/** HTTP requests to the physical uploads/ directory."

19. Apache Maven & pom.xml (Build Tool & Dependency Management)

"I used Apache Maven as the build automation tool. The pom.xml file declares all project dependencies (spring-boot-starter-data-jpa, spring-boot-starter-webmvc, mysql-connector-j, lombok). Maven downloads these JARs from Maven Central, compiles Java source code, runs tests, and packages the application into an executable JAR file."

20. Project Lombok (Boilerplate Code Reduction)

"I included Lombok in my pom.xml as an optional dependency. Lombok can auto-generate getter/setter methods, constructors, toString(), equals(), and hashCode() methods at compile time using annotations like @Getter, @Setter, @Data. In this project, I wrote getters/setters manually in the entities, but Lombok is configured and available for future use."

Section 2 — Project File Structure & What Each File Does

Complete directory tree with the purpose and role of every single file:

```

bookstore/
  pom.xml                                # Maven build config & dependencies
  src/main/resources/
    application.properties               # DB credentials, Hibernate, server port
    static/
      index.html                        # Customer storefront (browse, cart, order)
      admin.html                        # Admin dashboard (add, delete, upload)
  src/main/java/com/bookhub/bookstore/
    BookstoreApplication.java           # Main entry point + uploads config
    entity/
      Book.java                         # @Entity -> `book` table
      CartItem.java                    # @Entity -> `cart_item` table (FK to book)
      Order.java                       # @Entity -> `orders` table
    dto/
      ApiResponse.java                 # Generic DTO: {success, message, data}
    repository/
      BookRepository.java              # JpaRepository<Book, Long>
      CartRepository.java              # JpaRepository<CartItem, Long>
      OrderRepository.java             # JpaRepository<Order, Long> + findById
    service/
      BookService.java                 # CRUD logic for books
      CartService.java                 # Add-to-cart with stock validation
      OrderService.java                # Checkout: stock deduction, order creation
    controller/
      BookController.java              # REST: /books endpoints + file upload
      CartController.java              # REST: /cart endpoints
      OrderController.java             # REST: /orders endpoints
    exception/
      GlobalExceptionHandler.java      # @ControllerAdvice error handler
    uploads/                           # Stored cover images (served via HTTP)

```

File	Layer	What It Does
pom.xml	Build	Declares dependencies (Spring Boot, JPA, MySQL driver, Lombok), Java 21 version, and Maven build plugins.
application.properties	Config	Configures MySQL JDBC URL (localhost:3306/bookstore_db), username/password, Hibernate ddl-auto=update, show-sql=true, and server port 8080.
BookstoreApplication.java	Entry	Contains main() which boots the Spring container. Also declares a @Bean WebMvcConfigurer to serve uploaded images from the uploads/ directory over HTTP.
Book.java	Entity	JPA @Entity class mapping to the 'book' MySQL table. Fields: id (PK, auto-increment), title, author, price, quantity (stock), imageUrl.
CartItem.java	Entity	JPA @Entity mapping to 'cart_item' table. Has a @ManyToOne relationship to Book via foreign key column 'book_id'. Fields: id, quantity, book.
Order.java	Entity	JPA @Entity mapping to 'orders' table (not 'order' — SQL reserved word!). Fields: id, totalPrice, userId (non-nullable).
ApiResponse.java	DTO	Generic wrapper class ApiResponse<T> with fields: boolean success, String message, T data. Returned by every controller endpoint.
BookRepository.java	Repository	Interface extending JpaRepository<Book, Long>. Inherits findAll(), findById(), save(), deleteById() — zero SQL needed.
CartItemRepository.java	Repository	Interface extending JpaRepository<CartItem, Long>. Uses findAll(), save(), deleteAll().
OrderRepository.java	Repository	Extends JpaRepository<Order, Long>. Adds custom derived query method: List<Order> findById(String userId) which auto-generates SELECT * FROM orders WHERE user_id = ?.

BookService.java	Service	Business logic for catalog CRUD. Methods: getAllBooks(), saveBook(Book), deleteBook(Long id).
CartService.java	Service	Validates book existence and stock availability before adding to cart. Methods: addToCart(id, qty), getCart(), clearCart().
OrderService.java	Service	Orchestrates checkout: validates cart, checks stock, calculates total, decrements inventory, clears cart, saves order. Methods: placeOrder(userId), getOrdersByUser(userId).
BookController.java	Controller	REST endpoints: GET /books, POST /books, DELETE /books/{id}, POST /books/upload. Maps HTTP requests to BookService methods.
CartController.java	Controller	REST endpoints: POST /cart/{id}/{qty}, GET /cart. Maps to CartService methods.
OrderController.java	Controller	REST endpoints: POST /orders/{userId}, GET /orders/{userId}. Maps to OrderService methods.
GlobalExceptionHandler.java	Exception	@ControllerAdvice intercepting RuntimeException (HTTP 400) and Exception (HTTP 500) across all controllers.
index.html	Frontend	Customer storefront. JS functions: loadBooks(), addToCart(), loadCart(), placeOrder(), loadOrders(), toggleCart(). Uses Fetch API.
admin.html	Frontend	Admin dashboard. JS functions: addBook() (with image upload), loadBooks(), deleteBook(). Image preview via URL.createObjectURL().

Section 3 — How the Application Accesses MySQL (Complete Database Pipeline)

This section explains, step by step, the entire chain from application.properties configuration down to raw SQL execution on MySQL:

Step 1: application.properties configures the DataSource

```
spring.datasource.url=jdbc:mysql://localhost:3306/bookstore_db
spring.datasource.username=root
spring.datasource.password=root
```

At startup, Spring Boot reads these properties and creates a HikariDataSource bean. HikariCP uses mysql-connector-j (the JDBC driver) to open real TCP socket connections to the MySQL server process listening on localhost:3306.

Step 2: Hibernate scans @Entity classes and creates/updates tables (ddl-auto=update)

Hibernate inspects Book.java, CartItem.java, and Order.java. For each @Entity, it checks if the corresponding MySQL table exists. If not, it executes CREATE TABLE. If the table exists but columns differ, it executes ALTER TABLE. This is controlled by spring.jpa.hibernate.ddl-auto=update.

```
-- Hibernate auto-generated DDL:
CREATE TABLE book (id BIGINT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), author VARCHAR(255), price DOUBLE, quantity INT, image_url VARCHAR(255));
CREATE TABLE cart_item (id BIGINT AUTO_INCREMENT PRIMARY KEY, quantity INT, book_id BIGINT, FOREIGN KEY (book_id) REFERENCES book(id));
CREATE TABLE orders (id BIGINT AUTO_INCREMENT PRIMARY KEY, total_price DOUBLE, user_id VARCHAR(255) NOT NULL);
```

Step 3: Spring Data JPA generates Repository proxy implementations

Spring scans interfaces like BookRepository extends JpaRepository<Book, Long>. At runtime, Spring creates a dynamic proxy class (backed by SimpleJpaRepository) that implements all CRUD methods. No developer implementation class is needed!

Step 4: Query execution lifecycle (what happens when code calls repo.findAll())

```
Java Code: bookRepo.findAll()
|
v
Spring Data JPA proxy -> Hibernate EntityManager
|
v
Hibernate generates SQL: SELECT b.id, b.title, b.author, b.price, b.quantity, b.image_url FROM book
b
|
v
HikariCP borrows a pooled JDBC Connection (TCP socket to MySQL)
|
v
MySQL server executes query, returns ResultSet over TCP
|
v
Hibernate maps ResultSet rows -> List<Book> Java objects
|
v
HikariCP returns Connection to pool (reused for next request)
|
v
Service returns List<Book> -> Controller wraps in ApiResponse -> Jackson serializes to JSON
```

Step 5: Derived Query Methods (how findByUserId works)

```
// In OrderRepository.java:  
List<Order> findByUserId(String userId);  
  
// Spring Data JPA parses the method name:  
//   findBy -> SELECT * FROM orders WHERE  
//   UserId -> user_id = ?  
// Auto-generated SQL: SELECT * FROM orders WHERE user_id = ?
```

Spring Data JPA analyzes the method name at startup. It splits `findByUserId` into `findBy` (query keyword) + `UserId` (property name on the `Order` entity). It then generates the SQL query automatically. No JPQL, no native SQL, no manual implementation!

MEMORIZE THIS FOR YOUR INTERVIEW

Interview Summary: 'My application accesses MySQL through a 5-layer pipeline: **application.properties** configures the JDBC URL and credentials → **HikariCP** maintains a pool of TCP connections to MySQL → **Hibernate ORM** maps Java entities to SQL tables and auto-generates DDL → **Spring Data JPA** creates dynamic proxy implementations of my Repository interfaces → When my code calls methods like `findAll()` or `save()`, Hibernate generates the SQL query, borrows a HikariCP connection, executes the query on MySQL, maps the results back to Java objects, and returns the connection to the pool.'

Section 4 — How Frontend Connects to Backend (Complete HTTP Lifecycle)

This is **the most important answer** for your interview. Understand every step of how a button click in HTML reaches MySQL and returns data:

Complete Request-Response Lifecycle (every single step):

1. USER clicks 'Add to Cart' button in index.html
2. JavaScript function addToCart(id, stock) is invoked
3. Client-side validation: checks qty > 0 and qty <= stock
4. Browser executes: fetch('http://localhost:8080/cart/1/2', { method: 'POST' })
5. Browser creates HTTP POST request with headers and sends over TCP to localhost:8080
6. Embedded Tomcat receives the TCP connection and HTTP request
7. Tomcat passes request to Spring DispatcherServlet
8. DispatcherServlet consults HandlerMapping to find matching @RequestMapping
9. Finds: CartController.add(@PathVariable Long id, @PathVariable int qty)
10. Spring extracts path variables: id=1, qty=2 from URL /cart/1/2
11. CartController.add() calls CartService.addToCart(1, 2)
12. CartService calls bookRepo.findById(1L)
13. Spring Data JPA proxy -> Hibernate -> SQL: SELECT * FROM book WHERE id = 1
14. HikariCP borrows connection -> MySQL executes query -> returns ResultSet
15. Hibernate maps row to Book Java object
16. CartService validates: qty(2) <= book.getQuantity(10)? YES -> continues
17. CartService creates new CartItem, sets book and quantity
18. CartService calls cartRepo.save(item)
19. Hibernate -> SQL: INSERT INTO cart_item (book_id, quantity) VALUES (1, 2)
20. MySQL inserts row, returns generated ID
21. CartService returns CartItem to CartController
22. CartController wraps in: new ApiResponse<>(true, 'Added', cartItem)
23. Jackson ObjectMapper serializes ApiResponse to JSON string
24. Spring sets Content-Type: application/json, @CrossOrigin adds CORS headers
25. Tomcat sends HTTP 200 OK response with JSON body back to browser
26. Browser's fetch() Promise resolves with the Response object
27. JavaScript calls res.json() to parse JSON response
28. JavaScript displays alert('Added 2 x "Clean Code" to your cart!')
29. JavaScript calls loadBooks() and loadCart() to refresh the UI

FRONTEND-BACKEND CONNECTION — MEMORIZE THIS

Interview Answer (memorize this): 'I connect the HTML/JS frontend and Spring Boot backend through REST APIs. The frontend uses the native JavaScript Fetch API to send HTTP requests (GET, POST, DELETE) to endpoints like /books, /cart/{id}/{qty}, and /orders/{userId}. The Spring Boot backend uses @RestController classes with @GetMapping, @PostMapping, and @DeleteMapping to receive requests. The controller delegates to a @Service class for business logic, which calls @Repository interfaces for database access via Spring Data JPA and Hibernate. All responses are wrapped in a standardized ApiResponse<T> DTO and serialized to JSON by Jackson. @CrossOrigin enables CORS so the browser accepts responses from the backend.'

Section 5 — File-by-File & Function-by-Function Code Breakdown

Every single file, every function, with the actual code and line-by-line explanation:

FILE 1: application.properties (src/main/resources/)

What it does: Central configuration file read by Spring Boot at startup. No Java code — just key-value pairs.

```
spring.datasource.url=jdbc:mysql://localhost:3306/bookstore_db # JDBC URL:
protocol:subprotocol://host:port/database
spring.datasource.username=root # MySQL login username
spring.datasource.password=root # MySQL login password
spring.jpa.open-in-view=false # Closes DB connections after service layer (prevents connection
leaks)
spring.jpa.hibernate.ddl-auto=update # Hibernate auto-creates/updates tables from @Entity classes
spring.jpa.show-sql=true # Prints all Hibernate-generated SQL to console for debugging
server.port=8080 # Embedded Tomcat listens on this TCP port
```

FILE 2: BookstoreApplication.java

What it does: The main() entry point that boots the entire application, and configures static resource serving for uploaded images.

```
@SpringBootApplication // = @Configuration + @EnableAutoConfiguration + @ComponentScan
public class BookstoreApplication {

    // FUNCTION: main(String[] args)
    // WHAT: The very first line executed by the JVM. Bootstraps Spring Boot.
    // HOW: SpringApplication.run() creates ApplicationContext, performs component scanning,
    //       auto-configures DataSource/Hibernate/Tomcat, and starts listening on port 8080.
    public static void main(String[] args) {
        SpringApplication.run(BookstoreApplication.class, args);
    }

    // FUNCTION: config()
    // WHAT: Returns a WebMvcConfigurer bean that maps HTTP /uploads/** to file:uploads/
    // WHY: By default Spring only serves from src/main/resources/static. This makes the
    //       external uploads/ directory publicly accessible so cover images load in browsers.
    // EXAMPLE: uploads/1740000000_cover.jpg -> http://localhost:8080/uploads/1740000000_cover.jpg
    @Bean
    public WebMvcConfigurer config() {
        return new WebMvcConfigurer() {
            @Override
            public void addResourceHandlers(ResourceHandlerRegistry registry) {
                registry.addResourceHandler("/uploads/**")
                    .addResourceLocations("file:uploads/");
            }
        };
    }
}
```

FILE 3: Book.java (entity package)

What it does: Defines the Book database table schema in Java. Each instance = one row in the 'book' MySQL table.

```

@Entity // Tells Hibernate: this class maps to a database table
public class Book {

    @Id // This field is the Primary Key
    @GeneratedValue(strategy = GenerationType.IDENTITY) // MySQL AUTO_INCREMENT
    private Long id; // PK, auto-generated. SQL: id BIGINT AUTO_INCREMENT PRIMARY KEY

    private String title; // Book title. SQL: title VARCHAR(255)
    private String author; // Author name. SQL: author VARCHAR(255)
    private double price; // Price in Rs. SQL: price DOUBLE
    private int quantity; // Warehouse stock. SQL: quantity INT
    private String imageUrl; // Cover image URL. SQL: image_url VARCHAR(255)

    public Book() {} // Required by JPA for reflection-based instantiation

    // Getter methods: getId(), getTitle(), getAuthor(), getPrice(), getQuantity(), getImageUrl()
    // -> Used by Jackson to serialize field values into JSON keys
    // -> Used by Hibernate to read field values for SQL queries

    // Setter methods: setId(), setTitle(), setAuthor(), setPrice(), setQuantity(), setImageUrl()
    // -> Used by Jackson to deserialize JSON into object fields
    // -> Used by Hibernate to populate fields from ResultSet data
    // -> Used by OrderService to decrement stock: book.setQuantity(stock - qty)
}

```

FILE 4: CartItem.java (entity package)

What it does: Represents one item in the shopping cart. Links to Book via a foreign key relationship.

```

@Entity // Maps to 'cart_item' table
public class CartItem {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private int quantity; // How many copies of this book the customer wants

    @ManyToOne // Relationship: Many CartItems can reference One Book
    @JoinColumn(name = "book_id") // Creates FK column 'book_id' in cart_item table
    private Book book; // Hibernate auto-JOINS: cartItem.getBook().getTitle()

    public CartItem() {} // Required by JPA
    // Getters: getId(), getQuantity(), getBook()
    // Setters: setId(), setQuantity(), setBook()
}

```

FILE 5: Order.java (entity package)

What it does: Stores permanent transaction records after checkout. Note the @Table(name='orders') to avoid SQL reserved word conflict.

```

@Entity
@Table(name = "orders") // CRITICAL: 'order' is SQL reserved keyword (used in ORDER BY)
                        // Without this, Hibernate would generate: CREATE TABLE order ...
                        // which causes a MySQL syntax error!

public class Order {

    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private double totalPrice; // Sum of (qty * price) for all cart items at checkout

    @Column(nullable = false) // Adds NOT NULL constraint in MySQL
    private String userId;    // Which customer placed this order (e.g., "user1")

    public Order() {}
    // Getters and Setters
}

```

FILE 6: ApiResponse.java (dto package)

What it does: Generic wrapper ensuring ALL endpoints return a consistent JSON structure.

```

public class ApiResponse<T> {

    private boolean success; // true = operation worked, false = error occurred
    private String message;  // Human-readable status ("Books fetched", "Stock not available")
    private T data;          // The actual payload: List<Book>, Book, CartItem, Order, or null

    public ApiResponse() {} // Jackson needs this for deserialization

    // Constructor used by every controller:
    public ApiResponse(boolean success, String message, T data) {
        this.success = success;
        this.message = message;
        this.data = data;
    }

    // Getters (isSuccess(), getMessage(), getData()) -> Jackson reads these to create JSON keys
    // Setters (setSuccess(), setMessage(), setData())

}

// EXAMPLE JSON OUTPUT:
// { "success": true, "message": "Books fetched", "data": [{"id":1,"title":"Clean Code",...}] }

```

FILES 7-9: Repository Interfaces (repository package)

What they do: Define the data access layer. Spring auto-generates implementations at runtime.

```
// BookRepository.java
public interface BookRepository extends JpaRepository<Book, Long> {
    // INHERITS (no code needed!):
    // List<Book> findAll()          -> SELECT * FROM book
    // Optional<Book> findById(Long) -> SELECT * FROM book WHERE id = ?
    // Book save(Book book)         -> INSERT INTO book ... (if new) or UPDATE book ... (if existing)
    // void deleteById(Long id)     -> DELETE FROM book WHERE id = ?
}

// CartRepository.java
public interface CartRepository extends JpaRepository<CartItem, Long> {
    // INHERITS:
    // List<CartItem> findAll()      -> SELECT * FROM cart_item (with eager fetch of Book)
    // CartItem save(CartItem)      -> INSERT INTO cart_item ...
    // void deleteAll()             -> DELETE FROM cart_item (clears entire cart)
}

// OrderRepository.java
public interface OrderRepository extends JpaRepository<Order, Long> {

    // CUSTOM DERIVED QUERY METHOD:
    List<Order> findByUserId(String userId);
    // Spring parses method name: findBy + UserId
    // Auto-generates: SELECT * FROM orders WHERE user_id = ?
    // No SQL, no JPQL, no @Query annotation needed!
}
```

FILE 10: BookService.java (service package)

What it does: Contains business logic for book catalog management. Injected into BookController.

```
@Service // Marks this as a Spring-managed service bean
public class BookService {

    private final BookRepository repo; // Injected by Spring

    // Constructor Injection: Spring provides BookRepository when creating this bean
    public BookService(BookRepository repo) {
        this.repo = repo;
    }

    // FUNCTION: getAllBooks()
    // WHAT: Returns every book in the catalog
    // SQL: SELECT * FROM book
    public List<Book> getAllBooks() {
        return repo.findAll();
    }

    // FUNCTION: saveBook(Book book)
    // WHAT: Inserts a new book or updates an existing one
    // SQL: INSERT INTO book (...) VALUES (...) [if id is null]
    //       UPDATE book SET ... WHERE id = ? [if id exists]
    public Book saveBook(Book book) {
        return repo.save(book);
    }

    // FUNCTION: deleteBook(Long id)
    // WHAT: Removes a book from catalog
    // SQL: DELETE FROM book WHERE id = ?
    public void deleteBook(Long id) {
        repo.deleteById(id);
    }
}
```

FILE 11: CartService.java (service package)

What it does: Validates book existence and stock availability, then saves cart items. Contains critical business rules.

```

@Service
public class CartService {

    private final CartRepository cartRepo;
    private final BookRepository bookRepo;

    public CartService(CartRepository cartRepo, BookRepository bookRepo) {
        this.cartRepo = cartRepo;
        this.bookRepo = bookRepo;
    }

    // FUNCTION: addToCart(Long id, int qty)
    // WHAT: Validates and adds a book to the cart
    // STEP 1: Look up book by ID. If not found -> throw RuntimeException
    // STEP 2: Check if qty exceeds warehouse stock. If yes -> throw RuntimeException
    // STEP 3: Create CartItem, link to Book, save to MySQL
    public CartItem addToCart(Long id, int qty) {
        Book book = bookRepo.findById(id) // SQL: SELECT * FROM book WHERE id = ?
            .orElseThrow(() -> new RuntimeException("Book not found")); // -> HTTP 400

        if (qty > book.getQuantity()) { // Business rule: can't exceed stock
            throw new RuntimeException("Stock not available"); // -> HTTP 400
        }

        CartItem item = new CartItem();
        item.setBook(book); // Sets foreign key book_id
        item.setQuantity(qty);
        return cartRepo.save(item); // SQL: INSERT INTO cart_item (book_id, quantity) VALUES (?, ?)
    }

    // FUNCTION: getCart()
    // SQL: SELECT * FROM cart_item JOIN book ON cart_item.book_id = book.id
    public List<CartItem> getCart() {
        return cartRepo.findAll();
    }

    // FUNCTION: clearCart()
    // SQL: DELETE FROM cart_item
    public void clearCart() {
        cartRepo.deleteAll();
    }
}

```

FILE 12: OrderService.java (service package) — THE MOST COMPLEX SERVICE

What it does: Orchestrates the entire checkout workflow: validates cart, re-checks stock, calculates total, decrements inventory, clears cart, and creates the order record.

```

@Service
public class OrderService {

    private final CartRepository cartRepo;
    private final BookRepository bookRepo;
    private final OrderRepository orderRepo;

    public OrderService(CartRepository cartRepo, BookRepository bookRepo, OrderRepository orderRepo) {
        this.cartRepo = cartRepo;
        this.bookRepo = bookRepo;
        this.orderRepo = orderRepo;
    }

    // FUNCTION: placeOrder(String userId)
    // THE MOST IMPORTANT FUNCTION IN THE ENTIRE APPLICATION
    // STEP 1: Fetch all cart items (SQL: SELECT * FROM cart_item)
    // STEP 2: If cart empty -> throw RuntimeException -> HTTP 400
    // STEP 3: Loop through each CartItem:
    //   a) Get the associated Book
    //   b) Verify stock hasn't been depleted since adding to cart
    //   c) Accumulate total price: total += qty * price
    //   d) Deduct stock: book.setQuantity(stock - qty)
    //   e) Save updated book (SQL: UPDATE book SET quantity = ? WHERE id = ?)
    // STEP 4: Clear cart (SQL: DELETE FROM cart_item)
    // STEP 5: Create Order with totalPrice and userId
    // STEP 6: Save order (SQL: INSERT INTO orders (total_price, user_id) VALUES (?, ?))
    public Order placeOrder(String userId) {
        List<CartItem> items = cartRepo.findAll(); // STEP 1
        if (items.isEmpty()) { // STEP 2
            throw new RuntimeException("Cart is empty");
        }
        double total = 0;
        for (CartItem item : items) { // STEP 3
            Book book = item.getBook();
            if (item.getQuantity() > book.getQuantity()) {
                throw new RuntimeException("Stock not available");
            }
            total += item.getQuantity() * book.getPrice(); // 3c
            book.setQuantity(book.getQuantity() - item.getQuantity()); // 3d
            bookRepo.save(book); // 3e
        }
        cartRepo.deleteAll(); // STEP 4
        Order order = new Order(); // STEP 5
        order.setTotalPrice(total);
        order.setUserId(userId);
        return orderRepo.save(order); // STEP 6
    }

    // FUNCTION: getOrdersByUser(String userId)
    // SQL: SELECT * FROM orders WHERE user_id = ?
    public List<Order> getOrdersByUser(String userId) {
        return orderRepo.findById(userId);
    }
}

```

FILE 13: BookController.java (controller package)

What it does: Exposes REST HTTP endpoints for book catalog management. Maps URLs to service methods.

```

@RestController          // = @Controller + @ResponseBody (returns JSON, not HTML views)
@RequestMapping("/books") // Base URL prefix: all endpoints start with /books
@CrossOrigin              // Adds CORS headers so browsers allow cross-origin requests
public class BookController {

    private final BookService service;
    public BookController(BookService service) { this.service = service; }

    // ENDPOINT: GET /books
    // WHAT: Returns all books in the catalog
    // CALLED BY: index.html loadBooks(), admin.html loadBooks()
    @GetMapping
    public ApiResponse<List<Book>> getBooks() {
        return new ApiResponse<>(true, "Books fetched", service.getAllBooks());
    }

    // ENDPOINT: POST /books
    // WHAT: Adds a new book. @RequestBody deserializes JSON body into Book object.
    // CALLED BY: admin.html addBook()
    @PostMapping
    public ApiResponse<Book> addBook(@RequestBody Book book) {
        return new ApiResponse<>(true, "Book added", service.saveBook(book));
    }

    // ENDPOINT: DELETE /books/{id}
    // WHAT: Deletes book by ID. @PathVariable extracts 'id' from URL.
    // CALLED BY: admin.html deleteBook(id)
    @DeleteMapping("/{id}")
    public ApiResponse<String> deleteBook(@PathVariable Long id) {
        service.deleteBook(id);
        return new ApiResponse<>(true, "Book deleted", null);
    }

    // ENDPOINT: POST /books/upload
    // WHAT: Receives binary image file, saves to uploads/ with timestamp prefix
    // CALLED BY: admin.html addBook() (Step 1 of 2)
    @PostMapping("/upload")
    public String uploadImage(@RequestParam("file") MultipartFile file) throws Exception {
        String folder = "uploads/";
        File dir = new File(folder);
        if (!dir.exists()) dir.mkdirs(); // Create directory if missing
        String fileName = System.currentTimeMillis() + "_" + file.getOriginalFilename();
        String path = folder + fileName;
        file.transferTo(new File(path)); // Write bytes to disk
        return "http://localhost:8080/" + path; // Return public URL
    }
}

```

FILE 14: CartController.java (controller package)

```

@RestController
@RequestMapping("/cart")
@CrossOrigin
public class CartController {

    private final CartService service;
    public CartController(CartService service) { this.service = service; }

    // ENDPOINT: POST /cart/{id}/{qty}
    // WHAT: Adds book to cart. Both id and qty come from URL path.
    // CALLED BY: index.html addToCart(id, stock)
    @PostMapping("/{id}/{qty}")
    public ApiResponse<CartItem> add(@PathVariable Long id, @PathVariable int qty) {
        return new ApiResponse<>(true, "Added", service.addToCart(id, qty));
    }

    // ENDPOINT: GET /cart
    // WHAT: Returns all current cart items with nested Book data
    // CALLED BY: index.html loadBooks() and loadCart()
    @GetMapping
    public List<CartItem> get() {
        return service.getCart();
    }
}

```

FILE 15: OrderController.java (controller package)

```

@RestController
@RequestMapping("/orders")
@CrossOrigin
public class OrderController {

    private final OrderService service;
    public OrderController(OrderService service) { this.service = service; }

    // ENDPOINT: POST /orders/{userId}
    // WHAT: Places order for given user. Triggers checkout workflow.
    // CALLED BY: index.html placeOrder()
    @PostMapping("/{userId}")
    public ApiResponse<Order> placeOrder(@PathVariable String userId) {
        return new ApiResponse<>(true, "Order placed", service.placeOrder(userId));
    }

    // ENDPOINT: GET /orders/{userId}
    // WHAT: Fetches order history for given user.
    // CALLED BY: index.html loadOrders()
    @GetMapping("/{userId}")
    public ApiResponse<List<Order>> getOrders(@PathVariable String userId) {
        return new ApiResponse<>(true, "Orders fetched", service.getOrdersByUser(userId));
    }
}

```

FILE 16: GlobalExceptionHandler.java (exception package)

What it does: Intercepts all unhandled exceptions across every controller and converts them into clean JSON error responses.

```

@ControllerAdvice // AOP: applies to ALL @RestController classes automatically
public class GlobalExceptionHandler {

    // FUNCTION: handleRuntimeException(RuntimeException ex)
    // CATCHES: "Book not found", "Stock not available", "Cart is empty"
    // RETURNS: HTTP 400 Bad Request + ApiResponse(false, ex.getMessage(), null)
    @ExceptionHandler(RuntimeException.class)
    public ResponseEntity<ApiResponse<Object>> handleRuntimeException(RuntimeException ex) {
        ApiResponse<Object> response = new ApiResponse<>(false, ex.getMessage(), null);
        return new ResponseEntity<>(response, HttpStatus.BAD_REQUEST); // 400
    }

    // FUNCTION: handleGeneralException(Exception ex)
    // CATCHES: Any unexpected error (NullPointerException, DB connection failure, etc.)
    // RETURNS: HTTP 500 Internal Server Error + generic safe message
    @ExceptionHandler(Exception.class)
    public ResponseEntity<ApiResponse<Object>> handleGeneralException(Exception ex) {
        ApiResponse<Object> response = new ApiResponse<>(false, "Something went wrong", null);
        return new ResponseEntity<>(response, HttpStatus.INTERNAL_SERVER_ERROR); // 500
    }
}

```

FILE 17: index.html (Customer Storefront) — All JavaScript Functions

What it does: Single-page customer interface for browsing books, managing cart, placing orders, and viewing order history. Uses Fetch API.

```

// GLOBAL VARIABLE:
const userId = "user1"; // Hardcoded user ID (in production: from JWT/session)

// FUNCTION 1: loadBooks()
// WHAT: Fetches book catalog AND cart items, merges them, renders HTML cards
// HTTP: GET /books + GET /cart (parallel requests)
async function loadBooks() {
  let res = await fetch('http://localhost:8080/books'); // GET all books
  let data = await res.json(); // Parse ApiResponse
  let cartRes = await fetch('http://localhost:8080/cart'); // GET cart items
  let cart = await cartRes.json();
  let cartMap = {};
  cart.forEach(c => { cartMap[c.book.id] = c.quantity; });
  data.data.forEach(b => {
    let cartQty = cartMap[b.id] || 0; // How many already in cart
    // Renders: image, title, author, price, stock, 'In Cart: X', qty input, Add button
  });
  document.getElementById('books').innerHTML = html;
}

// FUNCTION 2: addToCart(id, stock)
// WHAT: Validates input, sends POST to add item to cart, refreshes UI
// HTTP: POST /cart/{id}/{qty}
async function addToCart(id, stock) {
  let qty = parseInt(document.getElementById('qty-' + id).value);
  if (!qty || qty <= 0) return alert('Enter valid quantity');
  if (qty > stock) return alert('Exceeds stock!');
  let res = await fetch('http://localhost:8080/cart/${id}/${qty}', { method: 'POST' });
  let result = await res.json();
  if (res.ok) { alert('Added!'); loadBooks(); loadCart(); }
  else { alert('Error: ' + result.message); }
}

// FUNCTION 3: loadCart()
// WHAT: Fetches cart items, calculates total, updates cart drawer + badge
// HTTP: GET /cart
async function loadCart() {
  let res = await fetch('http://localhost:8080/cart');
  let cart = await res.json();
  let total = 0, count = 0;
  cart.forEach(item => {
    total += item.book.price * item.quantity;
    count += item.quantity;
  });
  document.getElementById('cart').innerHTML = html;
  document.getElementById('total').innerText = 'Total: Rs' + total;
  document.getElementById('cartCount').innerText = count;
}

```

index.html — Functions 4-6 (continued)

```
// FUNCTION 4: placeOrder()
// WHAT: Triggers checkout. Backend decrements stock, clears cart, saves order.
// HTTP: POST /orders/{userId}
async function placeOrder() {
  let res = await fetch(`http://localhost:8080/orders/${userId}`, { method: 'POST' });
  let data = await res.json();
  if (!res.ok) return alert('Checkout Failed: ' + data.message);
  alert(`Order #${data.data.id} placed! Total: Rs${data.data.totalPrice}`);
  document.getElementById('cartPanel').classList.remove('active');
  loadBooks(); loadCart(); loadOrders(); // Refresh everything
}

// FUNCTION 5: loadOrders()
// WHAT: Fetches and displays order history for current user
// HTTP: GET /orders/{userId}
async function loadOrders() {
  let res = await fetch(`http://localhost:8080/orders/${userId}`);
  let data = await res.json();
  data.data.forEach(o => { /* Render: Order #ID, userId, totalPrice */ });
  document.getElementById('orders').innerHTML = html;
}

// FUNCTION 6: toggleCart()
// WHAT: Slides cart panel in/out using CSS class toggle
function toggleCart() {
  let panel = document.getElementById('cartPanel');
  panel.classList.toggle('active'); // CSS: right: -420px -> right: 0
  if (panel.classList.contains('active')) loadCart();
}

// INITIALIZATION: runs when page loads
loadBooks(); loadCart(); loadOrders();
```

FILE 18: admin.html (Admin Dashboard) — All JavaScript Functions

```
// IMAGE PREVIEW EVENT LISTENER
// WHAT: Shows instant preview when admin selects an image file
// 
document.getElementById('imageFile').onchange = function(e) {
  let preview = document.getElementById('preview');
  preview.src = URL.createObjectURL(e.target.files[0]); // Client-side blob URL
  preview.style.display = 'block'; // Make visible
};

// 
// FUNCTION: addBook()
// WHAT: Uploads image + saves book to catalog (2-step process)
// HTTP STEP 1: POST /books/upload (multipart/form-data) -> returns image URL
// HTTP STEP 2: POST /books (application/json) -> saves Book entity
// 
async function addBook() {
  // Read form inputs
  let title = document.getElementById('title').value;
  let author = document.getElementById('author').value;
  let price = parseFloat(document.getElementById('price').value);
  let quantity = parseInt(document.getElementById('quantity').value);
  let imageUrl = document.getElementById('imageUrl').value;

  // Validate required fields
  if (!title || !author || isNaN(price) || isNaN(quantity)) return alert('Fill all fields');

  // STEP 1: Upload image file (if selected)
  if (fileInput.files.length > 0) {
    let formData = new FormData();
    formData.append('file', fileInput.files[0]);
    let res = await fetch('http://localhost:8080/books/upload', { method: 'POST', body: formData });
    imageUrl = await res.text(); // e.g. 'http://localhost:8080/uploads/1740000000_cover.jpg'
  }

  // STEP 2: Save book entity
  let book = { title, author, price, quantity, imageUrl };
  let response = await fetch('http://localhost:8080/books', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(book)
  });
  let json = await response.json();
  if (response.ok && json.success) {
    alert(`Book "${json.data.title}" added!`);
    loadBooks(); // Refresh grid
  }
}

// 
// FUNCTION: loadBooks() – same as index.html but with Delete buttons
// HTTP: GET /books
// 
async function loadBooks() { /* Renders cards with Delete button */ }

// 
// FUNCTION: deleteBook(id)
// WHAT: Confirms and deletes book from catalog
// HTTP: DELETE /books/{id}
// 
async function deleteBook(id) {
  if (!confirm('Delete this book?')) return;
  let res = await fetch('http://localhost:8080/books/' + id, { method: 'DELETE' });
  if (res.ok) { alert('Deleted!'); loadBooks(); }
}

// INITIALIZATION
loadBooks();
```

Section 6 — End-to-End Workflow: What Happens When Each Action Is Selected

A. Page Load (Customer Opens index.html)

1. Browser loads `http://localhost:8080/index.html` from Spring's `static/` folder
2. HTML/CSS renders. `<script>` block executes immediately.
3. Calls `loadBooks()` -> `fetch GET /books` -> `BookController.getBooks()` -> `BookService.getAllBooks()` -> `BookRepository.findAll()` -> SQL: `SELECT * FROM book` -> returns `List<Book>` -> `ApiResponse JSON`
4. Calls `loadCart()` -> `fetch GET /cart` -> `CartController.get()` -> `CartService.getCart()` -> `CartRepository.findAll()` -> SQL: `SELECT * FROM cart_item JOIN book` -> returns `List<CartItem>` JSON
5. Calls `loadOrders()` -> `fetch GET /orders/user1` -> `OrderController.getOrders('user1')` -> `OrderService.getOrdersByUser('user1')` -> `OrderRepository.findById('user1')` -> SQL: `SELECT * FROM orders WHERE user_id = 'user1'`
6. JavaScript builds HTML cards from JSON data and injects into DOM

B. Customer Clicks 'Add to Cart'

1. `onclick='addToCart(1, 10)'` triggers JavaScript function
2. Reads `qty` from input `#qty-1` (e.g., `qty=2`)
3. Client validation: `qty > 0?` YES. `qty <= stock(10)?` YES.
4. `fetch('http://localhost:8080/cart/1/2', { method: 'POST' })`
5. Tomcat -> `DispatcherServlet` -> `CartController.add(id=1, qty=2)`
6. `CartService.addToCart(1, 2)`:
 - a) `bookRepo.findById(1)` -> SQL: `SELECT * FROM book WHERE id = 1` -> Book found
 - b) `qty(2) > book.quantity(10)?` NO -> proceed
 - c) Create `CartItem(book=Book#1, quantity=2)`
 - d) `cartRepo.save(item)` -> SQL: `INSERT INTO cart_item (book_id, quantity) VALUES (1, 2)`
7. Returns `ApiResponse(true, 'Added', CartItem)` -> JSON -> HTTP 200 OK
8. Frontend: `alert('Added!')`, calls `loadBooks()` + `loadCart()` to refresh UI

C. Customer Clicks 'Place Order'

1. `onclick='placeOrder()'` triggers JavaScript function
2. `fetch('http://localhost:8080/orders/user1', { method: 'POST' })`
3. Tomcat -> `DispatcherServlet` -> `OrderController.placeOrder('user1')`
4. `OrderService.placeOrder('user1')`:
 - a) `cartRepo.findAll()` -> SQL: `SELECT * FROM cart_item` -> gets 1 item: `CartItem(book=Book#1, qty=2)`
 - b) `items.isEmpty()`? NO -> proceed
 - c) For `CartItem(Book#1, qty=2)`:
 - `item.qty(2) > book.qty(10)?` NO -> proceed
 - `total += 2 * 499.0 = 998.0`
 - `book.setQuantity(10 - 2 = 8)` -> `bookRepo.save(book)` -> SQL: `UPDATE book SET quantity = 8 WHERE id = 1`
 - d) `cartRepo.deleteAll()` -> SQL: `DELETE FROM cart_item`
 - e) Create `Order(totalPrice=998.0, userId='user1')`
 - f) `orderRepo.save(order)` -> SQL: `INSERT INTO orders (total_price, user_id) VALUES (998.0, 'user1')`
5. Returns `ApiResponse(true, 'Order placed', Order#1)` -> JSON -> HTTP 200
6. Frontend: `alert('Order #1 placed! Total: Rs998')`, closes cart drawer
7. Calls `loadBooks()` (stock now shows 8), `loadCart()` (badge shows 0), `loadOrders()` (shows Order #1)

D. Admin Adds a Book (admin.html)

1. Admin fills: title='Clean Code', author='Robert Martin', price=499, quantity=10
2. Admin selects cover image file -> onchange preview renders instantly via `URL.createObjectURL()`
3. Admin clicks 'Add Book' -> `addBook()` executes
4. STEP 1 – Image Upload:
 - a) `new FormData().append('file', imageFile)`
 - b) `fetch POST /books/upload` with body: `formData (multipart/form-data)`
 - c) `BookController.uploadImage()`: creates uploads/ dir, saves file as `1740000000_cover.jpg`
 - d) Returns text: `'http://localhost:8080/uploads/1740000000_cover.jpg'`
5. STEP 2 – Save Book:
 - a) Builds JSON: `{title, author, price, quantity, imageUrl}`
 - b) `fetch POST /books` with `Content-Type: application/json`
 - c) `BookController.addBook()` -> `@RequestBody` deserializes JSON into Book object
 - d) `BookService.saveBook(book)` -> `BookRepository.save(book)` -> SQL: `INSERT INTO book (title, author, price, quantity, image_url) VALUES (...)`
6. Returns `ApiResponse(true, 'Book added', Book#1)` -> JSON -> HTTP 200
7. Frontend: `alert('Book added!')`, clears form, calls `loadBooks()` to refresh grid

E. Admin Deletes a Book

1. Admin clicks 'Delete' on book card -> `deleteBook(1)` executes
2. `confirm('Delete this book?')` -> User clicks OK
3. `fetch('http://localhost:8080/books/1', { method: 'DELETE' })`
4. `BookController.deleteBook(1)` -> `BookService.deleteBook(1)` -> `BookRepository.deleteById(1)`
5. SQL: `DELETE FROM book WHERE id = 1`
6. Returns `ApiResponse(true, 'Book deleted', null)` -> HTTP 200
7. Frontend: `alert('Deleted!')`, calls `loadBooks()` to remove card from grid

Section 7 — Master REST API Reference Table & SQL Query Map

Endpoint	Controller.Method()	Service.Method()	SQL Executed	Called From
GET /books	BookController.getBooks()	BookService.getAllBooks()	SELECT * FROM book	index + admin
POST /books	BookController.addBook()	BookService.saveBook()	INSERT INTO book ...	admin
DELETE /books/{id}	BookController.deleteBook()	BookService.deleteBook()	DELETE FROM book WHERE id=?	admin
POST /books/upload	BookController.uploadImage()	(filesystem I/O)	(no SQL)	admin
POST /cart/{id}/{qty}	CartController.add()	CartService.addToCart()	SELECT book; INSERT cart_item	index
GET /cart	CartController.get()	CartService.getCart()	SELECT * FROM cart_item JOIN book	index
POST /orders/{userId}	OrderController.placeOrder()	OrderService.placeOrder()	SELECT cart; UPDATE book; DELETE cart; INSERT orders	index
GET /orders/{userId}	OrderController.getOrders()	OrderService.getOrdersByUser()	SELECT * FROM orders WHERE user_id=?	index

Section 8 — CRUD Operations Explained With Code

CRUD = Create, Read, Update, Delete. Here is how each operation works in BookHub:

Operation	HTTP Method	Example Endpoint	Java Code	SQL
CREATE	POST	POST /books	repo.save(book)	INSERT INTO book ...
READ (all)	GET	GET /books	repo.findAll()	SELECT * FROM book
READ (one)	GET	GET /books/{id}	repo.findById(id)	SELECT * FROM book WHERE id=?
UPDATE	PUT/POST	POST /books	repo.save(book) [if id exists]	UPDATE book SET ... WHERE id=?
DELETE	DELETE	DELETE /books/{id}	repo.deleteById(id)	DELETE FROM book WHERE id=?

Section 9 — Exception Handling Pipeline

What happens when something goes wrong — traced from service to browser:

Example: Customer tries to add 100 copies of a book that only has 10 in stock

1. Frontend: `fetch('http://localhost:8080/cart/1/100', { method: 'POST' })`
2. `CartController.add(id=1, qty=100) -> CartService.addToCart(1, 100)`
3. `CartService: bookRepo.findById(1) -> Book found (stock = 10)`
4. `CartService: 100 > 10? YES -> throw new RuntimeException('Stock not available')`
5. Exception bubbles up from Service -> Controller -> `DispatcherServlet`
6. `GlobalExceptionHandler` intercepts (because of `@ControllerAdvice + @ExceptionHandler`)
7. `handleRuntimeException(ex)` creates: `ApiResponse(false, 'Stock not available', null)`
8. Returns: `ResponseEntity` with HTTP 400 BAD REQUEST status
9. Tomcat sends response: `{ "success": false, "message": "Stock not available", "data": null }`
10. Frontend: `res.ok` is false -> `alert('Error: Stock not available')`

Exception Type	Thrown By	Message	HTTP Status
RuntimeException	<code>CartService.addToCart()</code>	'Book not found'	400 Bad Request
RuntimeException	<code>CartService.addToCart()</code>	'Stock not available'	400 Bad Request
RuntimeException	<code>OrderService.placeOrder()</code>	'Cart is empty'	400 Bad Request
RuntimeException	<code>OrderService.placeOrder()</code>	'Stock not available'	400 Bad Request
Exception (any other)	Any layer	'Something went wrong'	500 Internal Error

Section 10 — Interview Closing Summary & Golden Pitch

INTERVIEW GOLDEN PITCH — MEMORIZE THIS

30-Second Pitch (memorize this):

"BookHub is a full-stack e-commerce bookstore application built with **Java 21** and **Spring Boot 4.0.5** on the backend, **MySQL** as the relational database, and **HTML5/CSS3/JavaScript** on the frontend.

I structured the backend using a clean **3-tier layered architecture**: **Controllers** (REST endpoints) → **Services** (business logic) → **Repositories** (database access via Spring Data JPA/Hibernate).

The frontend connects to the backend using the **JavaScript Fetch API** to call RESTful endpoints. All responses are standardized using a generic **ApiResponse<T>** DTO. **@CrossOrigin** enables CORS, and a **GlobalExceptionHandler** with **@ControllerAdvice** converts all errors into clean JSON responses with proper HTTP status codes.

Key features include: **catalog CRUD management** with cover image upload, **shopping cart** with real-time stock validation, **checkout workflow** with automatic inventory deduction, and **order history** per user."

FULL SYSTEM ARCHITECTURE

Architecture Diagram:

```

[Browser: index.html / admin.html]
  | JavaScript Fetch API (HTTP GET/POST/DELETE + JSON)
  v
[Embedded Tomcat :8080] → [DispatcherServlet] → [@CrossOrigin CORS]
  |
  v
[@RestController] → parses @PathVariable, @RequestBody, @RequestParam
  |
  v
[@Service] → Business rules, validation, stock checks, price calculations
  |
  v
[JpaRepository] → Spring Data JPA dynamic proxy → Hibernate ORM
  |
  v
[HikariCP Connection Pool] → [mysql-connector-j JDBC Driver]
  |
  v
[MySQL Server :3306] → bookstore_db → Tables: book, cart_item, orders
  
```

WHAT EACH LAYER DOES — QUICK REFERENCE

Layer Responsibility Summary:

- **Controller Layer:** Receives HTTP requests, extracts parameters, delegates to Service, wraps response in ApiResponse, returns JSON.
- **Service Layer:** Contains ALL business logic — validation, stock checks, price calculations, cart operations. NEVER touches HTTP directly.
- **Repository Layer:** Pure data access. Extends JpaRepository. Zero business logic. Spring generates SQL implementation automatically.
- **Entity Layer:** Defines database table schema in Java. @Entity + @Id + @GeneratedValue = table with auto-increment PK.
- **DTO Layer:** ApiResponse<T> standardizes every API response format for frontend consumption.
- **Exception Layer:** @ControllerAdvice catches ALL errors globally, returns proper HTTP 400/500 status codes with JSON error messages.
- **Frontend Layer:** HTML/CSS/JS pages call backend REST APIs via fetch(), parse JSON responses, and dynamically update the DOM.